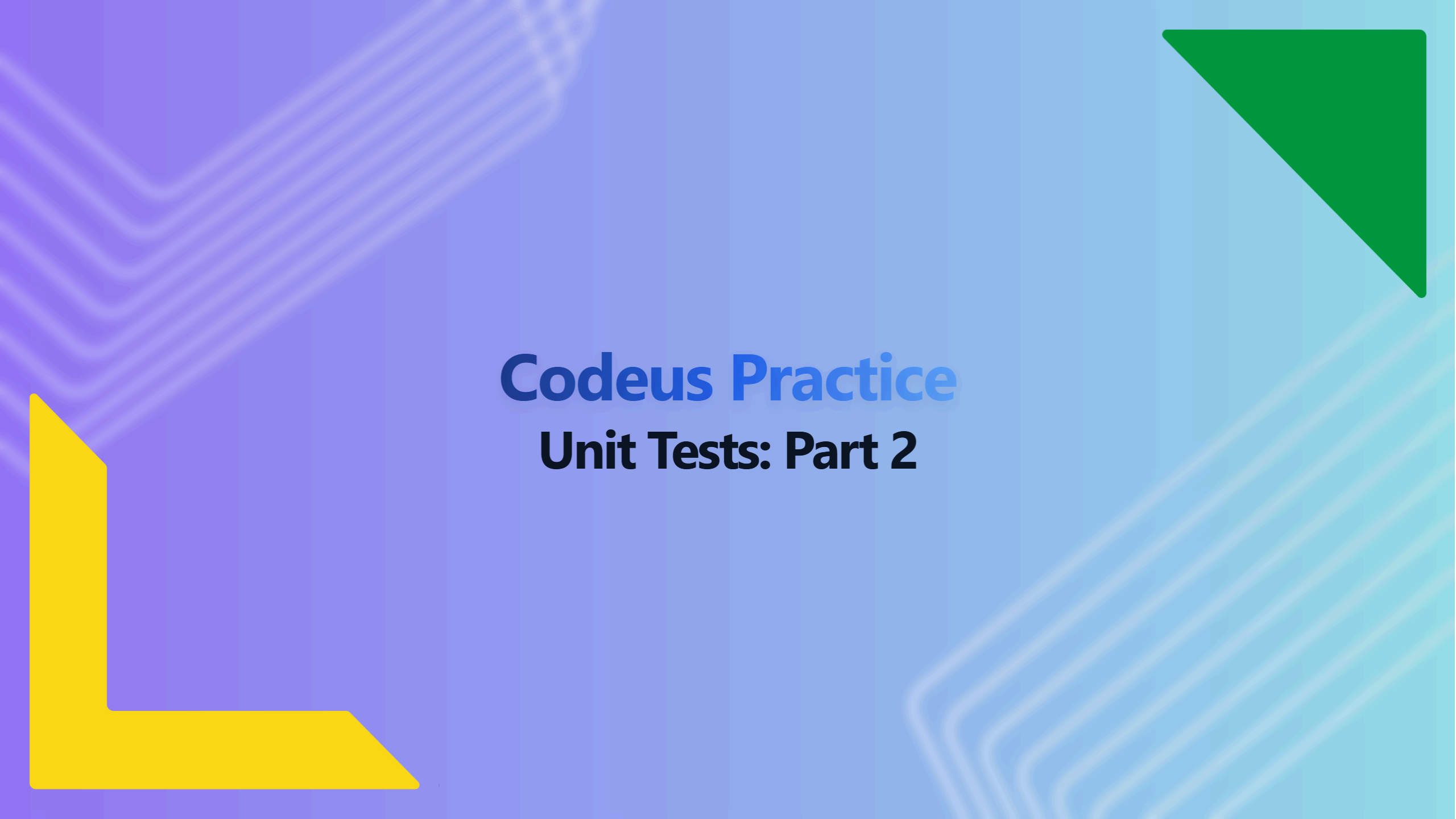




Codeus Practice

Unit Tests: Part 2

FIRST Principles

Fast

tests should run quickly enough to be executed constantly

Independent or Isolated

each test should stand on its own and focus on one unit

Repeatable

the same test should produce the same result on any machine and any run

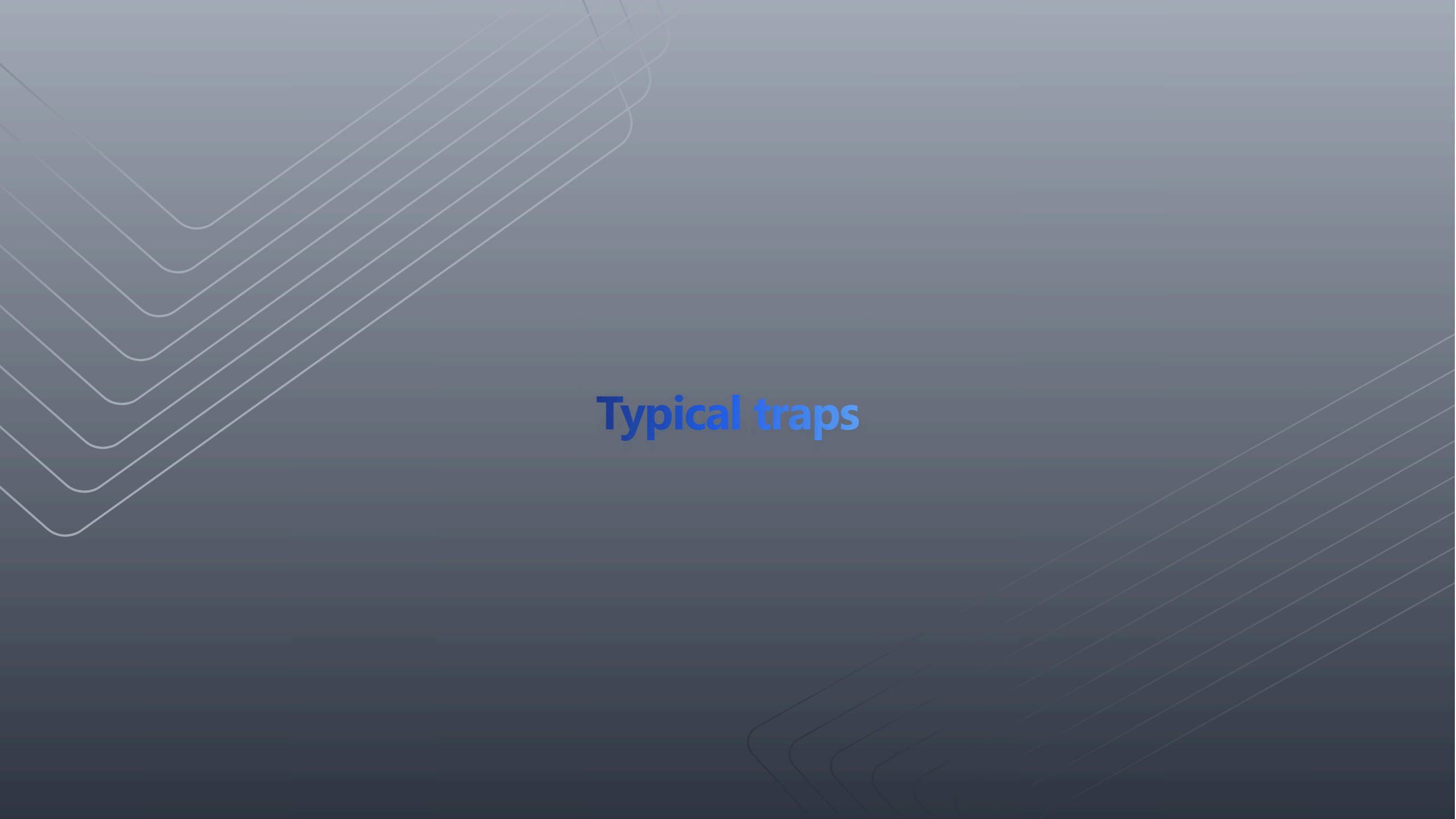
Self-validating

the test should clearly pass or fail without manual inspection

Thorough or Timely

cover important behaviors and edge cases, and write tests at the right time to guide design





Typical traps

Case 1

- Problem: Weak Assertions.
- A test is green, but it proves almost nothing about the returned quote.
- If the service returns the wrong number, this test still passes.

JAVA

```
@Test
void calculateQuote_returnsDefaultQuote() {
    BigDecimal total = service.calculateQuote("c-1", "Lviv", new BigDecimal("2"));

    assertNotNull(total);
}
```



Case 2

- Problem: Slow External Dependencies.
- This test reaches the network, so it is too slow and environment-sensitive for a unit test.
- The class under test is no longer isolated from infrastructure.

JAVA

```
@Test
void calculateQuote_returnsGoldTierQuoteWithLiveTax() {
    RoutePriceClient routePriceClient = new OnlineRoutePriceClient("https://carrier.example.com");
    CustomerTierService customerTierService = new CustomerTierService(customerRepository);
    QuoteEventPublisher quoteEventPublisher = new DeliveryQuoteEventSender();
    DeliveryQuoteService service = new DeliveryQuoteService(
        routePriceClient,
        customerTierService,
        quoteEventPublisher
    );

    BigDecimal total = service.calculateQuote("c-1", "Lviv", new BigDecimal("2"));

    assertEquals(new BigDecimal("17.00"), total);
}
```



Case 3

- Problem: Eager Tests.
- The test looks simple, but it verifies two behaviors at once: quote calculation and event publishing.
- If it fails, the bug could be in pricing, event creation, or event delivery state.

JAVA

```
@Test
void calculateQuote_sendsEventAndReturnsTotal() {
    RoutePriceClient routePriceClient = new LocalRoutePriceClient();
    CustomerTierService customerTierService = new CustomerTierService(customerRepository);
    DeliveryQuoteEventSender quoteEventPublisher = new DeliveryQuoteEventSender();
    DeliveryQuoteService service = new DeliveryQuoteService(routePriceClient, customerTierService, quoteEventPublisher);

    BigDecimal total = service.calculateQuote("c-1", "Lviv", new BigDecimal("2"));

    assertEquals(new BigDecimal("17.00"), total);
    assertEquals(1, quoteEventPublisher.sentEvents().size());
    QuoteCalculatedEvent event = quoteEventPublisher.sentEvents().get(0);
    assertEquals("c-1", event.customerId());
    assertEquals(new BigDecimal("17.00"), event.total());
}
```



Case 4

- Problem: Hidden Shared State.
- The second test passes only because the first test already filled the private cache.
- It even verifies that collaborators were not called, which hides the order dependency instead of fixing it.

```
JAVA
@TestMethodOrder(OrderAnnotation.class)
class DeliveryQuoteServiceTest {
    static RoutePriceClient routePriceClient = mock(OnlineRoutePriceClient.class);
    static CustomerTierService customerTierService = mock(CustomerTierService.class);
    static QuoteEventPublisher quoteEventPublisher = mock(DeliveryQuoteEventSender.class);
    static DeliveryQuoteService service =
        new DeliveryQuoteService(routePriceClient, customerTierService, quoteEventPublisher);

    @Test @Order(1)
    void calculateQuote_cachesGoldQuote() {
        when(routePriceClient.fetchBasePrice("Lviv")).thenReturn(new BigDecimal("10.00"));
        when(customerTierService.tierFor("c-1")).thenReturn("GOLD");
        assertEquals(new BigDecimal("17.00"), service.calculateQuote("c-1", "Lviv", new BigDecimal("2")));
    }

    @Test @Order(2)
    void calculateQuote_retrievesGoldQuoteFromCache() {
        reset(routePriceClient, customerTierService, quoteEventPublisher);
        assertEquals(new BigDecimal("17.00"), service.calculateQuote("c-1", "Lviv", new BigDecimal("2")));
        verifyNoInteractions(routePriceClient, customerTierService, quoteEventPublisher);
    }
}
```





Test Structure

Arrange / Act / Assert

JAVA

```
@Test
void calculateQuote_returnsGoldPrice_whenCustomerHasGoldTier() {
    when(routePriceClient.fetchBasePrice("Lviv")).thenReturn(new BigDecimal("10.00")); // Arrange
    when(customerTierService.tierFor("c-1")).thenReturn("GOLD"); // Arrange

    BigDecimal total = service.calculateQuote("c-1", "Lviv", new BigDecimal("2")); // Act

    assertEquals(new BigDecimal("17.00"), total); // Assert
}
```



Naming Pattern

Pattern

method_condition_expectedResult

Examples

withdraw_LeavingExactThresholdBalance_DoesNotSendAlert, convert_ThrowsException_WhenAmountIsNull



Task outline

3 Services

TransactionService, AccountService, InterestCalculator

Domain

Fintech - banking system

Goal

Write tests for the services from scratch





Thank you